Data Structures and Algorithms

Lecture 4 -

Linked List

Dept. of ICT, Faculty of Technology- UoR, 2026

List

What is a List?

It is a sequence of ELEMENTS.

Formally

it is a FINITE SEQUENCE of elements

Example list of integers or list of records.



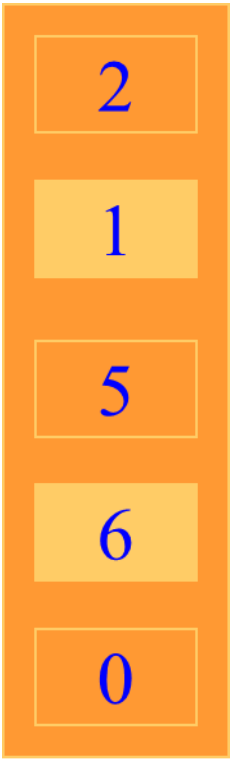
Storing similar data in memory, can be used either an array of a link list.

Implementation of List

There are 2 main ways :

1. Using Contiguous storage

an array in which the elements are physically next to one an another in adjacent memory locations.



2. Non Contiguous storage(Linked List)

Link list elements are not physically adjacent in memory

Disadvantage of Contiguous memory

The disadvantages of arrays are :

–The size of the array is fixed

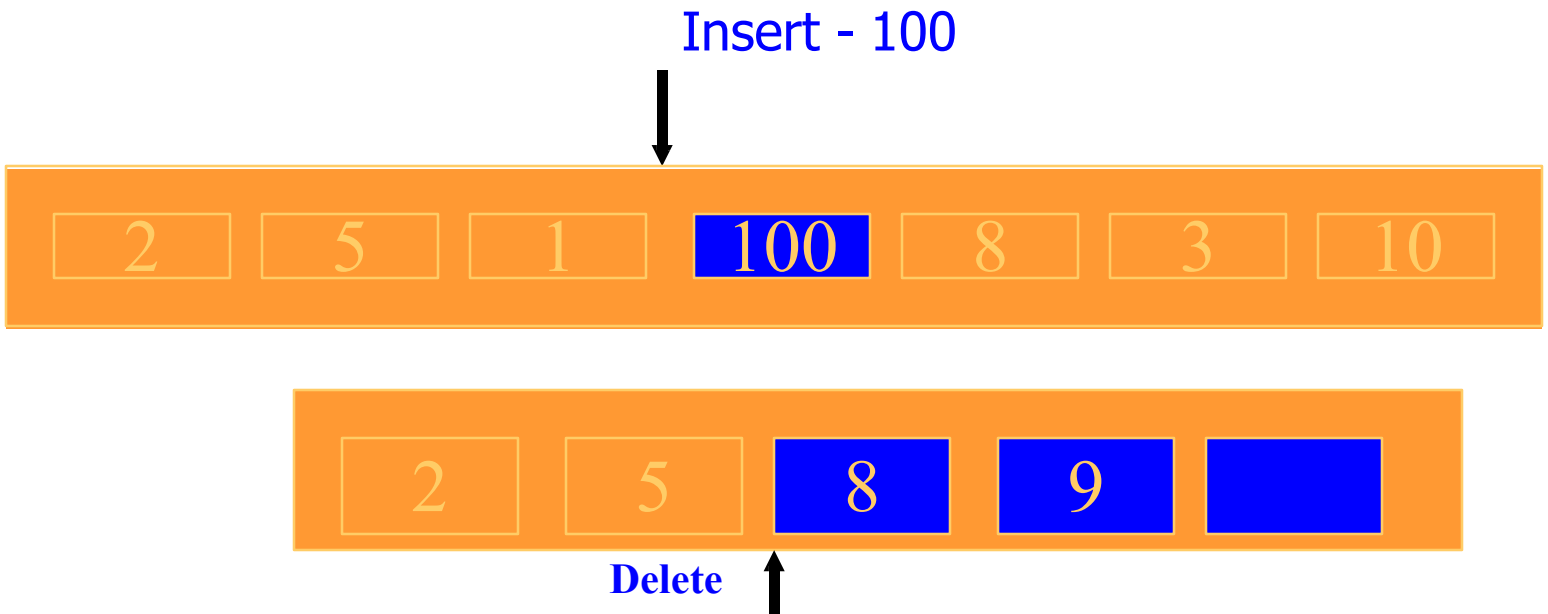
– Wastes space

–Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.

Disadvantage of Contiguous memory

Insertion in Position requires moving of elements 'DOWN' one position.

Deletion requires moving of elements 'UP' one position.



Linked List

Adv: Linked list eliminates the problem encountered in List.

What is a Linked List?

A linked list is a collection of **nodes**, where each node contains some data along with information about the next node.

How it works?

A linked list uses non-contiguous memory locations and hence requires each node to remember where the next node is

- Individual elements are stored " Somewhere" in Memory.
- The order of the elements is maintained by explicit links between them.

Arrays and Linked Lists

- Linked lists and arrays are similar since they both store collections of data.
- Array elements are easily assessable.
- Arrays are probably the most common data structure used to store collections of elements.
- In most languages, arrays are convenient to declare and provide easy syntax to access any element by its index number.

Arrays and Linked Lists

Linked list provides following two advantages over arrays

1) Dynamic size- can grow and shrink in size during its lifetime.
2) Ease of insertion/deletion – shifting of the node is not required.

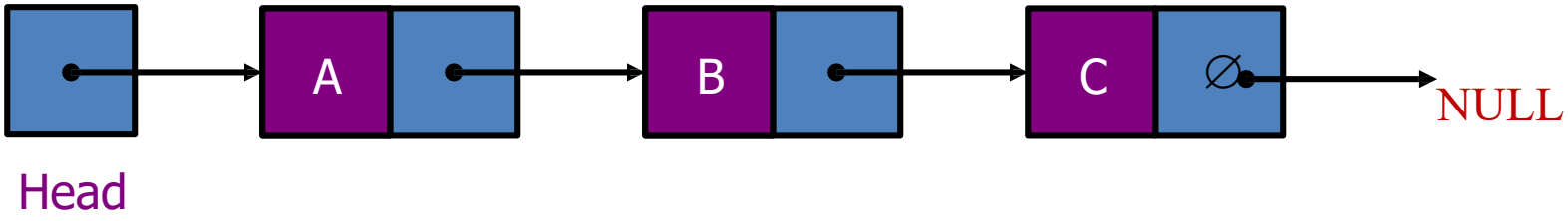
Linked lists have following drawbacks:

1) **Random access is not allowed.** We have to access elements sequentially starting from the first node. So we cannot do binary search with linked lists.
2) **Extra memory space for a pointer** is required with each element of the list.
3) Arrays have **better cache locality** that can make a pretty big difference in performance.

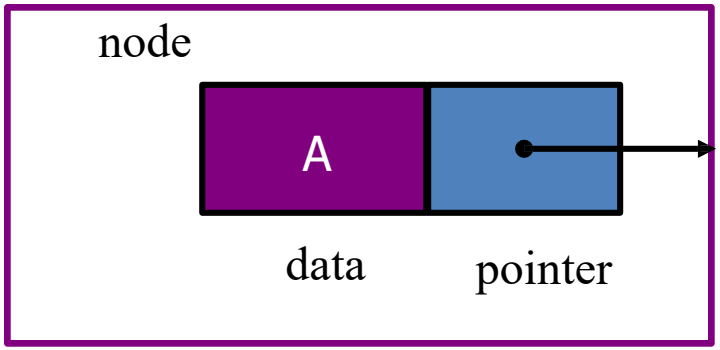
Linked lists

- A linked list is a dynamic data structure.
- Linked lists grow and shrink during their life time.
- Their maximum size does not need to be known in advance.
- The advantage of a list over an array occurs when it is necessary to insert or delete an element in the middle of a group of elements.
- Linked lists can handle insertions and deletions without shifting.

Linked Lists



- A *linked list* is a series of connected *nodes*
- Each node contains at least
  - A piece of data (any type)
  - Pointer to the next node in the list



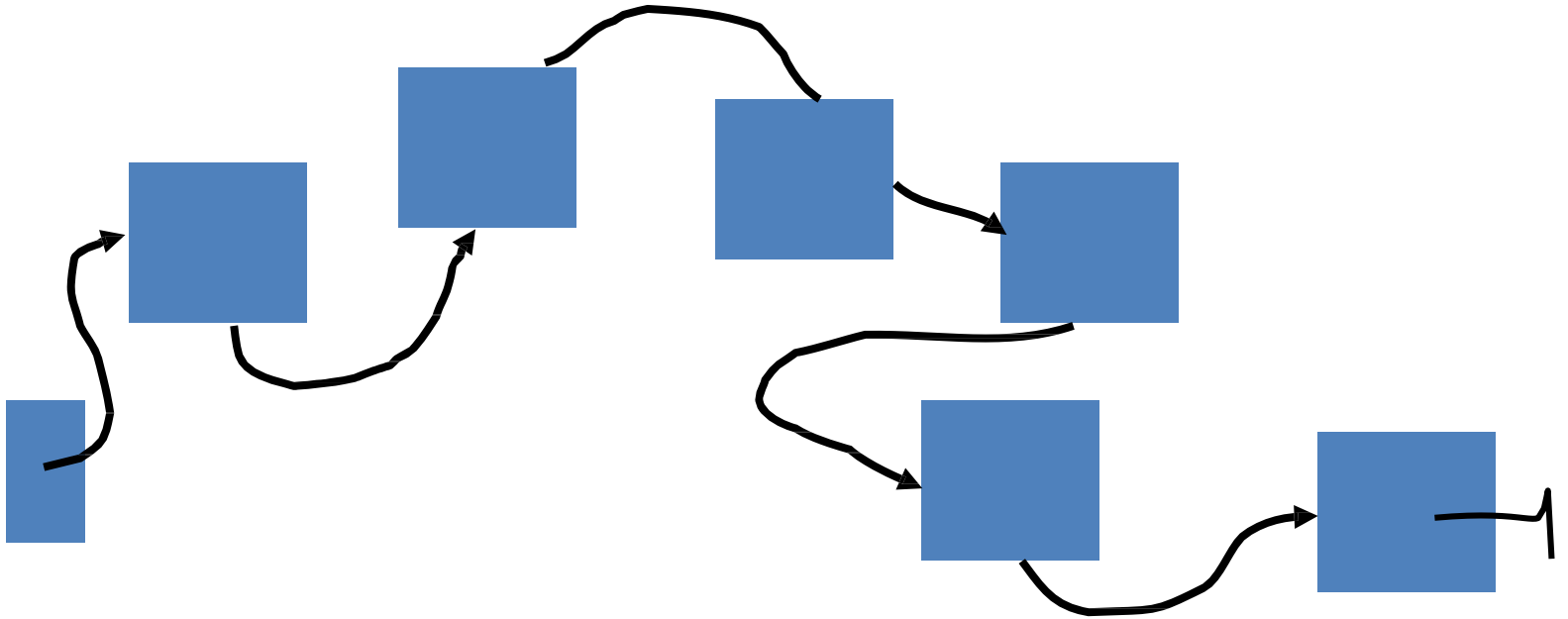
- *Head*: pointer to the first node
- The last node points to NULL

Linked Lists



- Linked List contains a link element called first/head.
- Each node carries a data field(s) and a link field called next.
- Each node is linked with its next link using its next link.
- Last node carries a link as null to mark the end of the list.

A Linked List

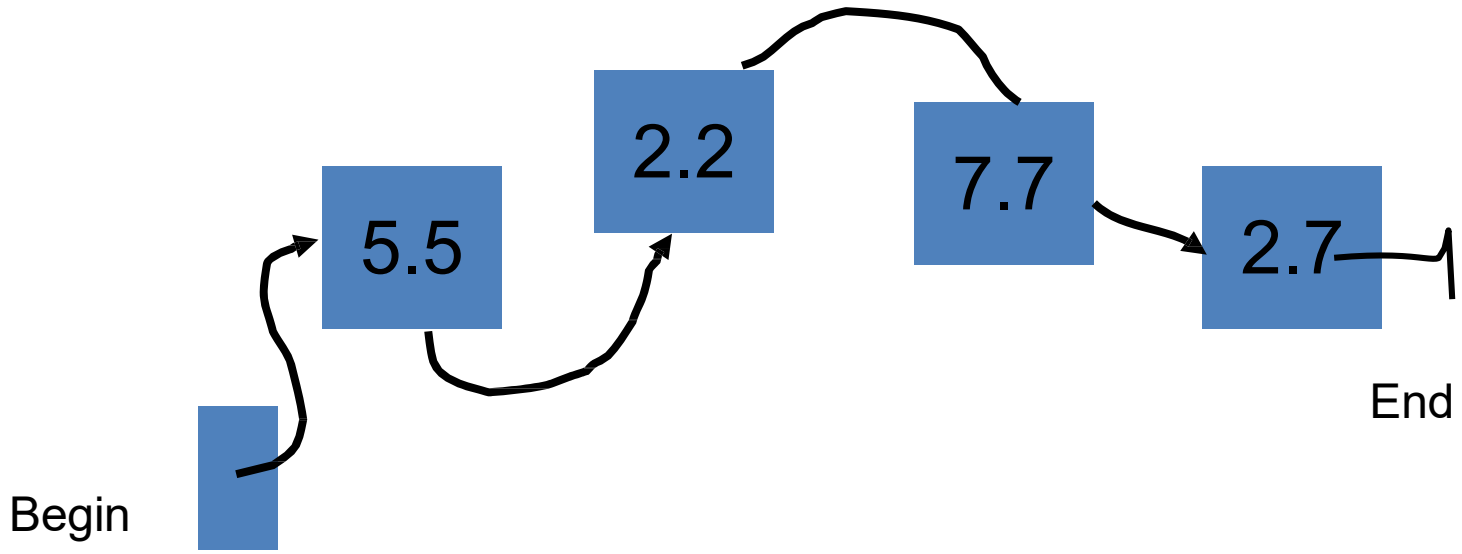


- A Linked List comprises of a set of data items (links) connected together

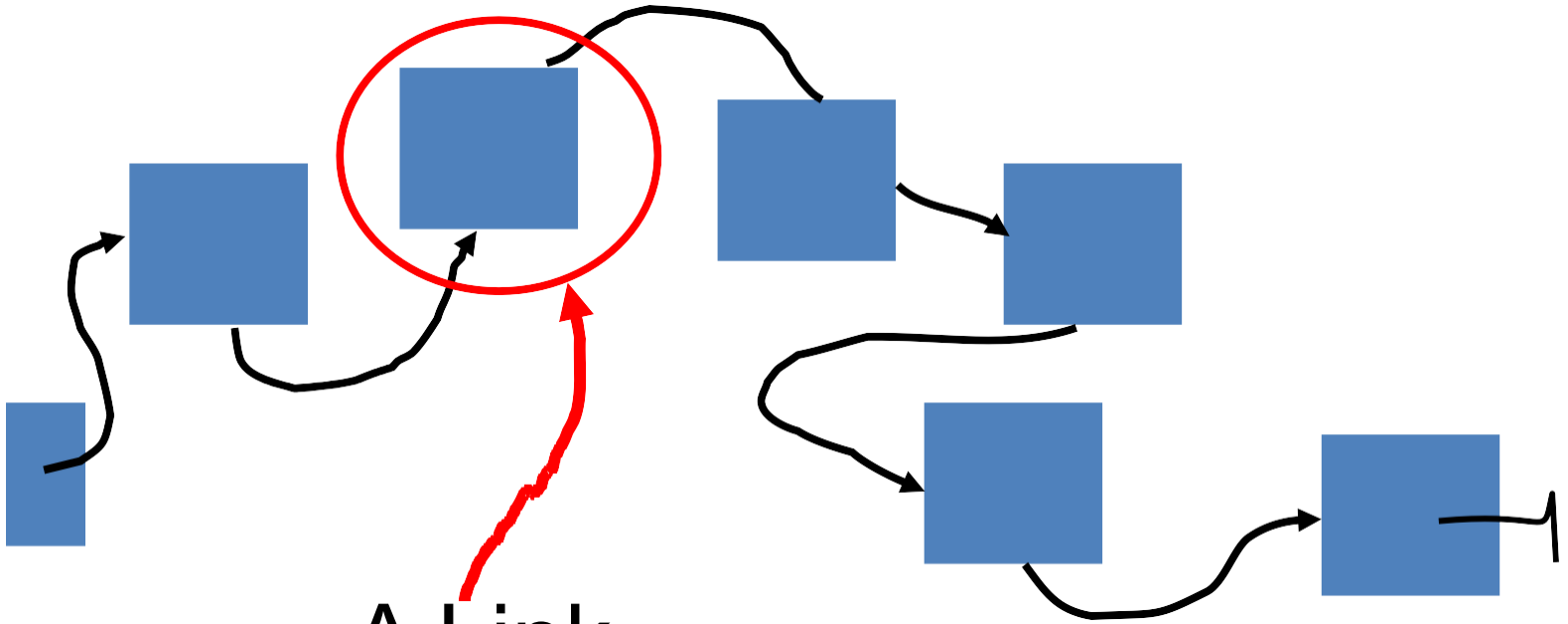
Array / Linked List

To store double values

0	5.5
1	2.2
2	7.7
3	2.7



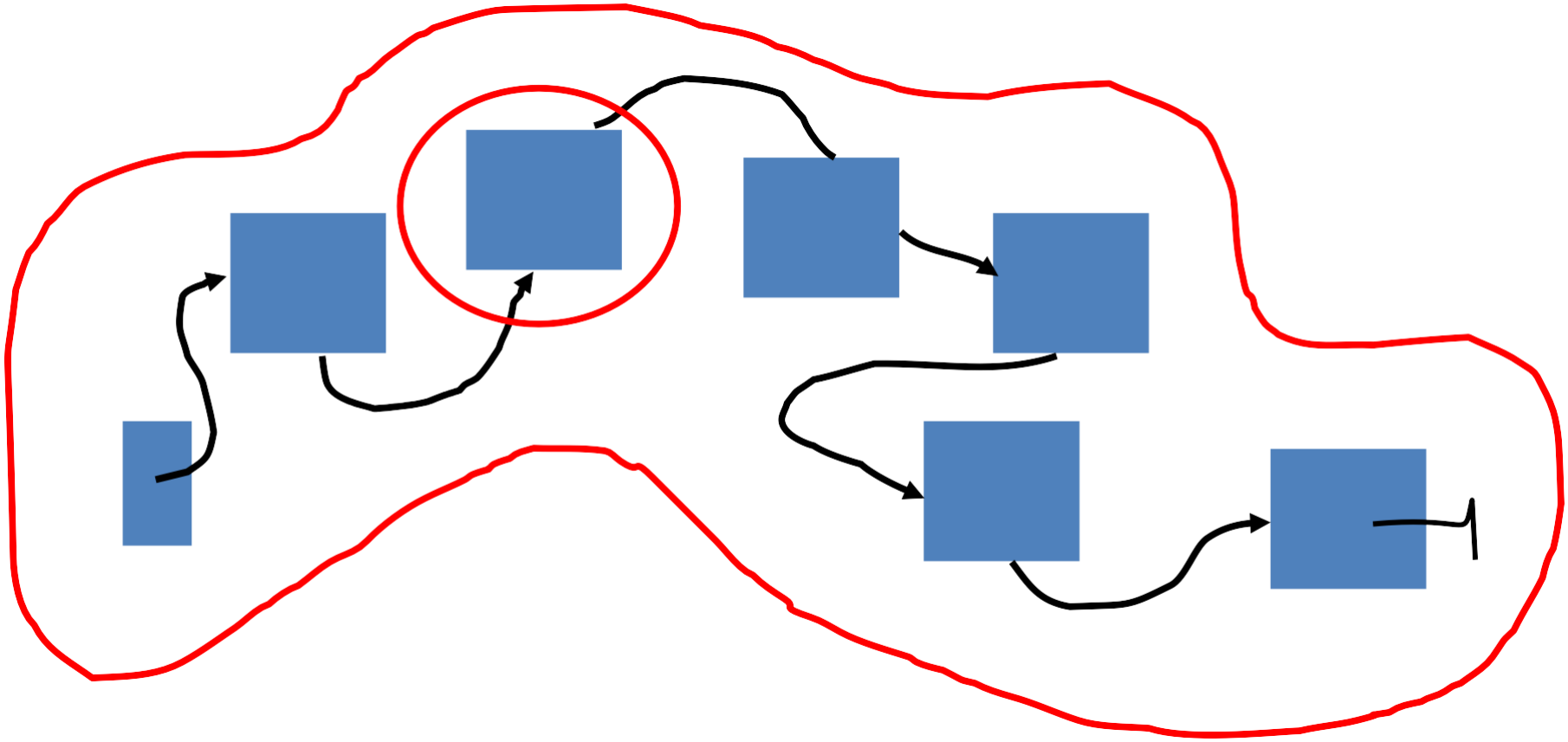
A Link



A Link

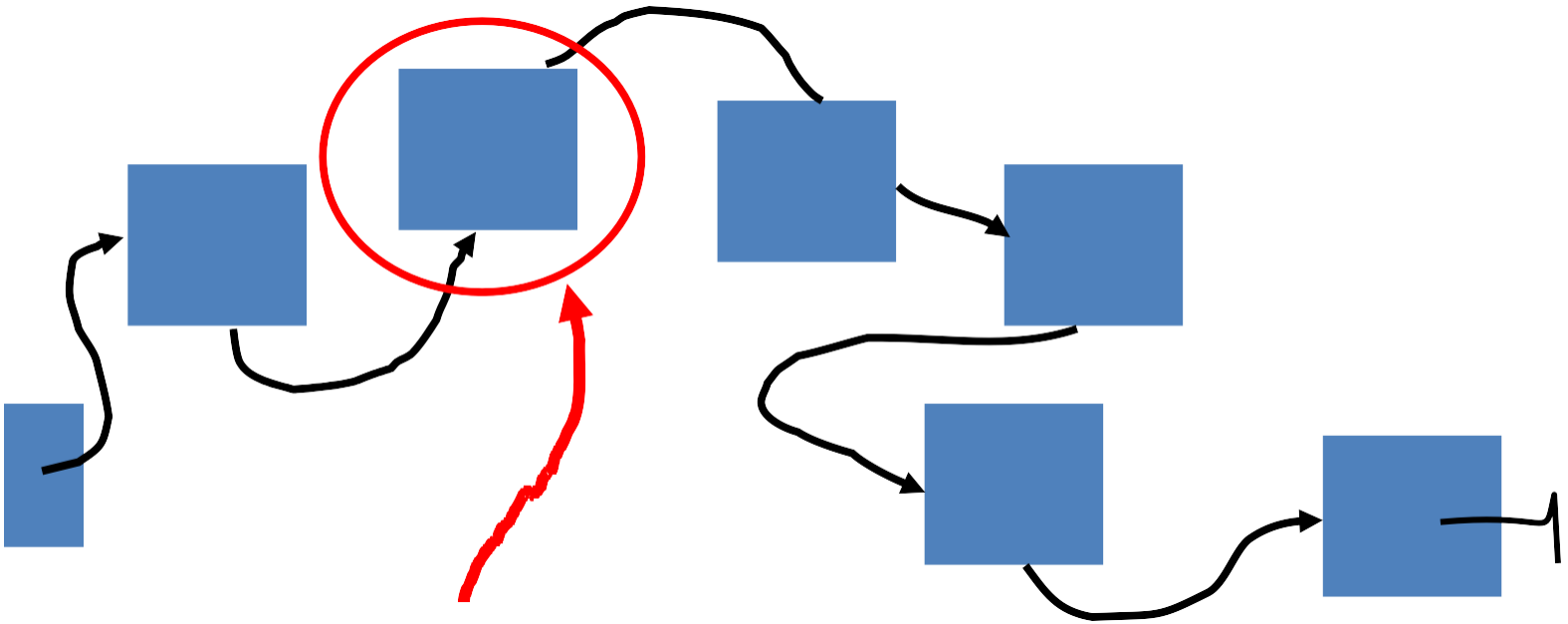
(data item)

A Linked List



A List of data items

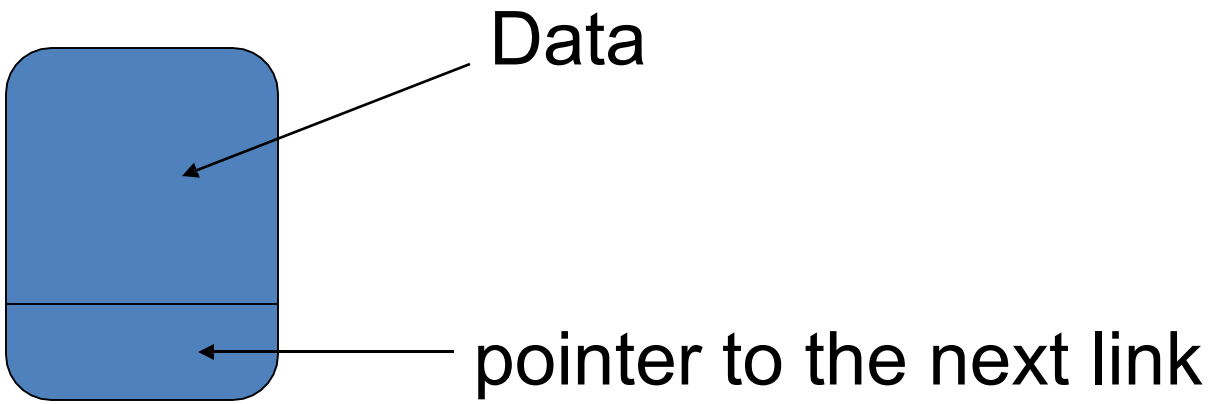
Handling the Linked List



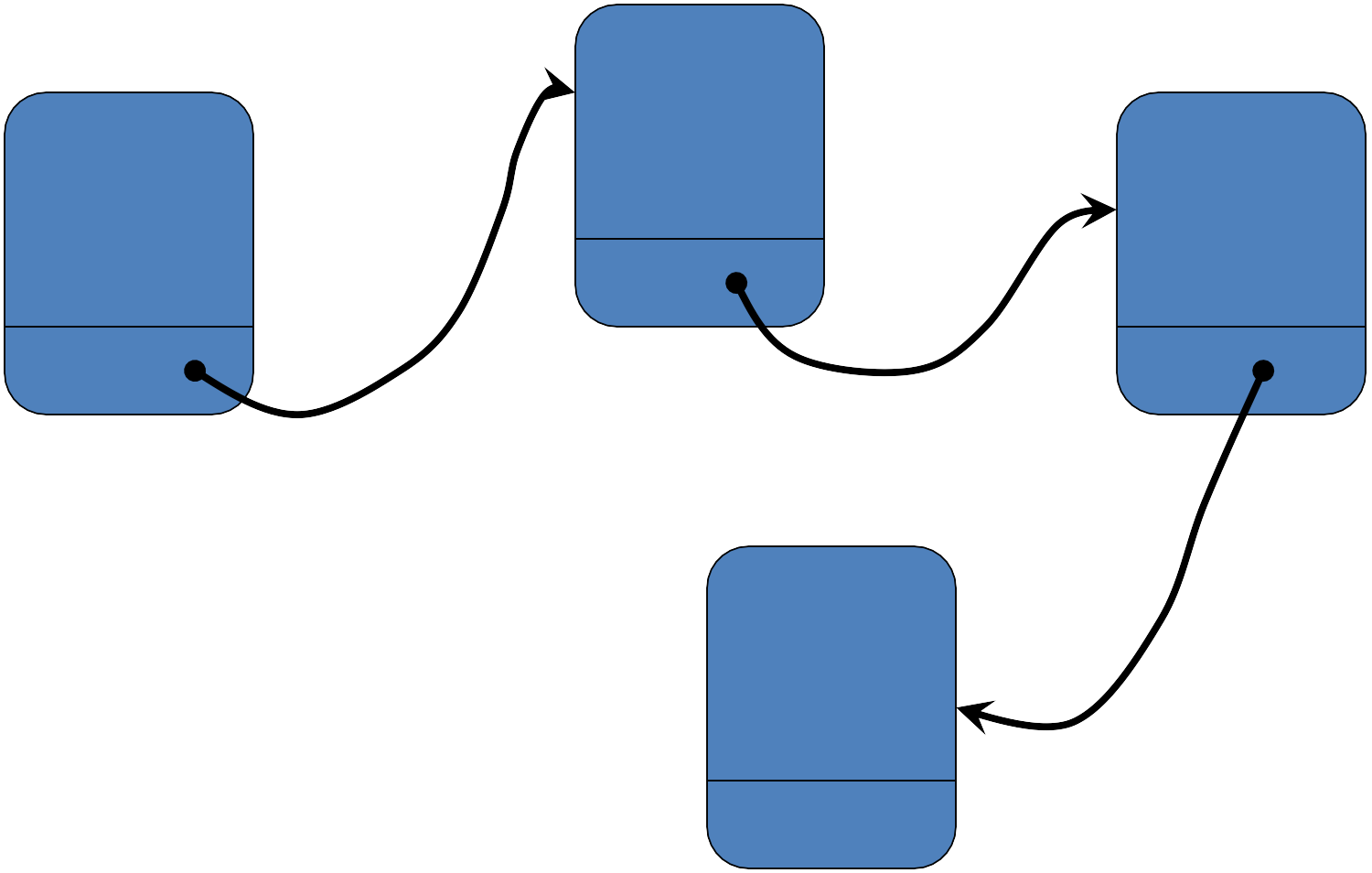
one structure for a link

```
struct Node
{
int Data;
struct Node * Next;
};
```

The Link

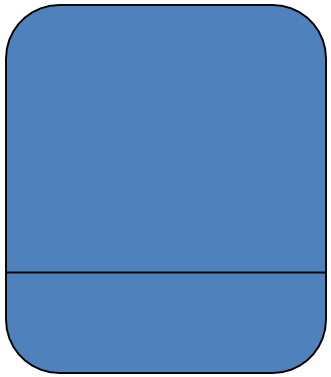


How the Links are Linked

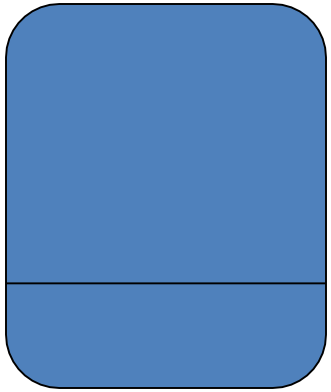


How the Links are Linked

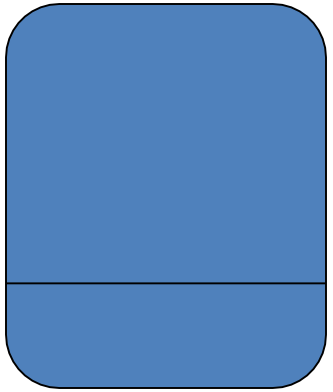
775AAA



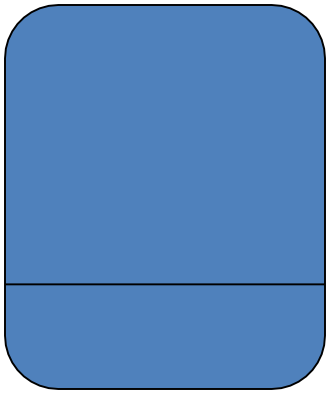
A257AA



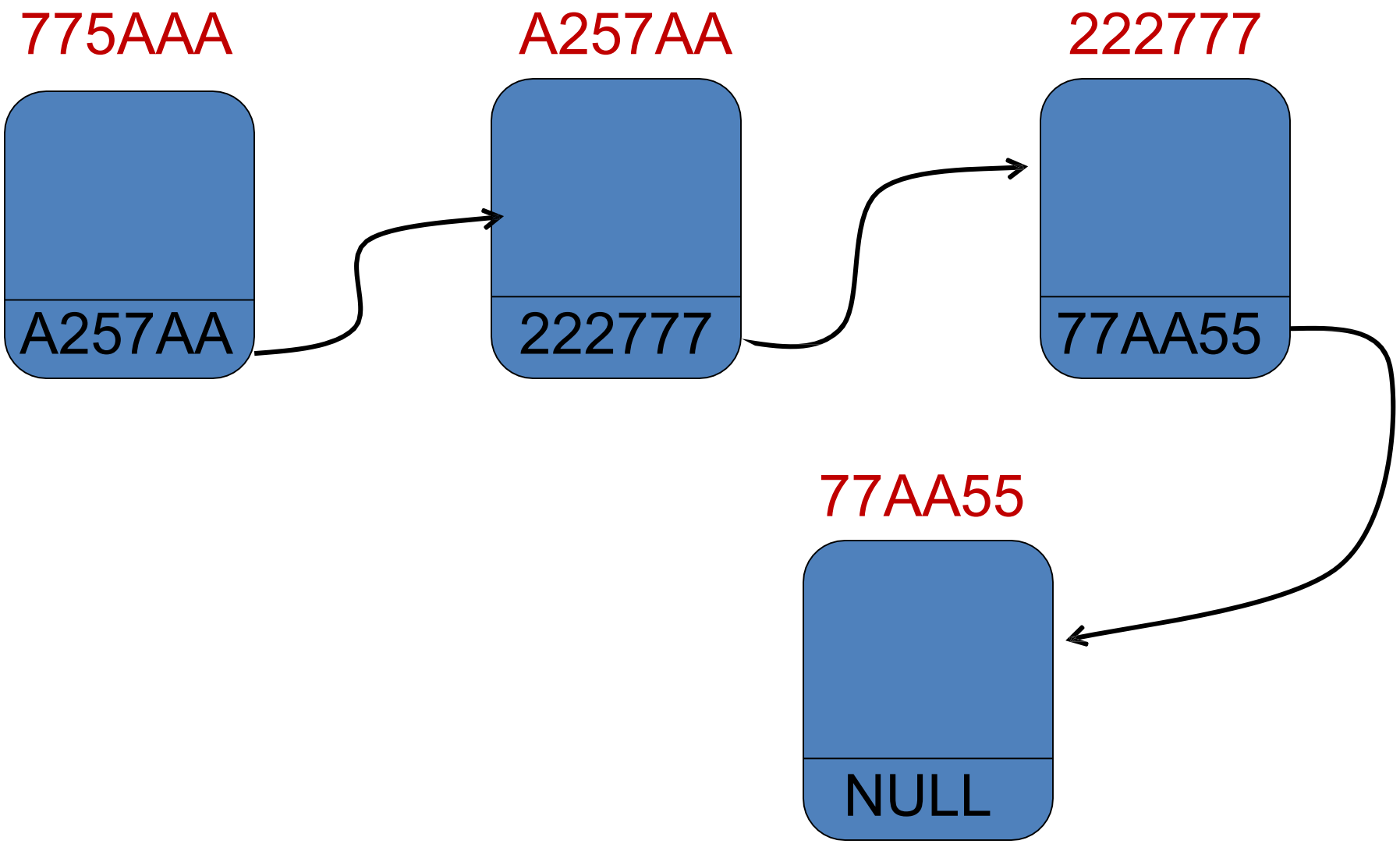
222777



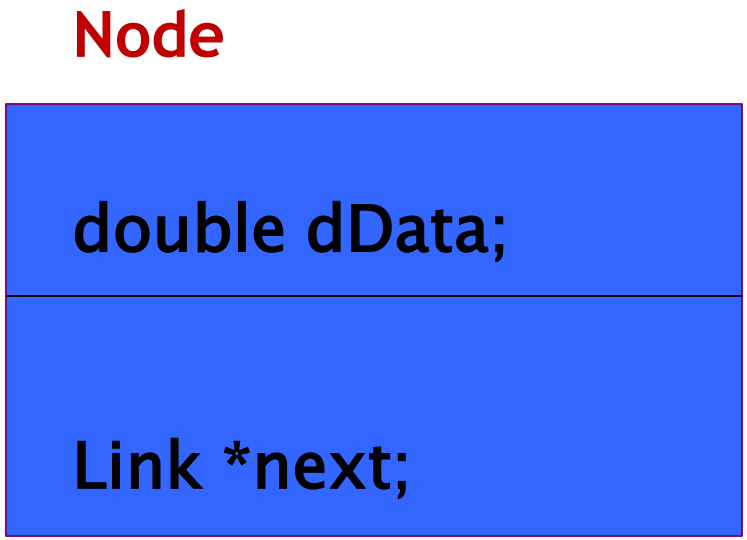
77AA55



How the Links are Linked

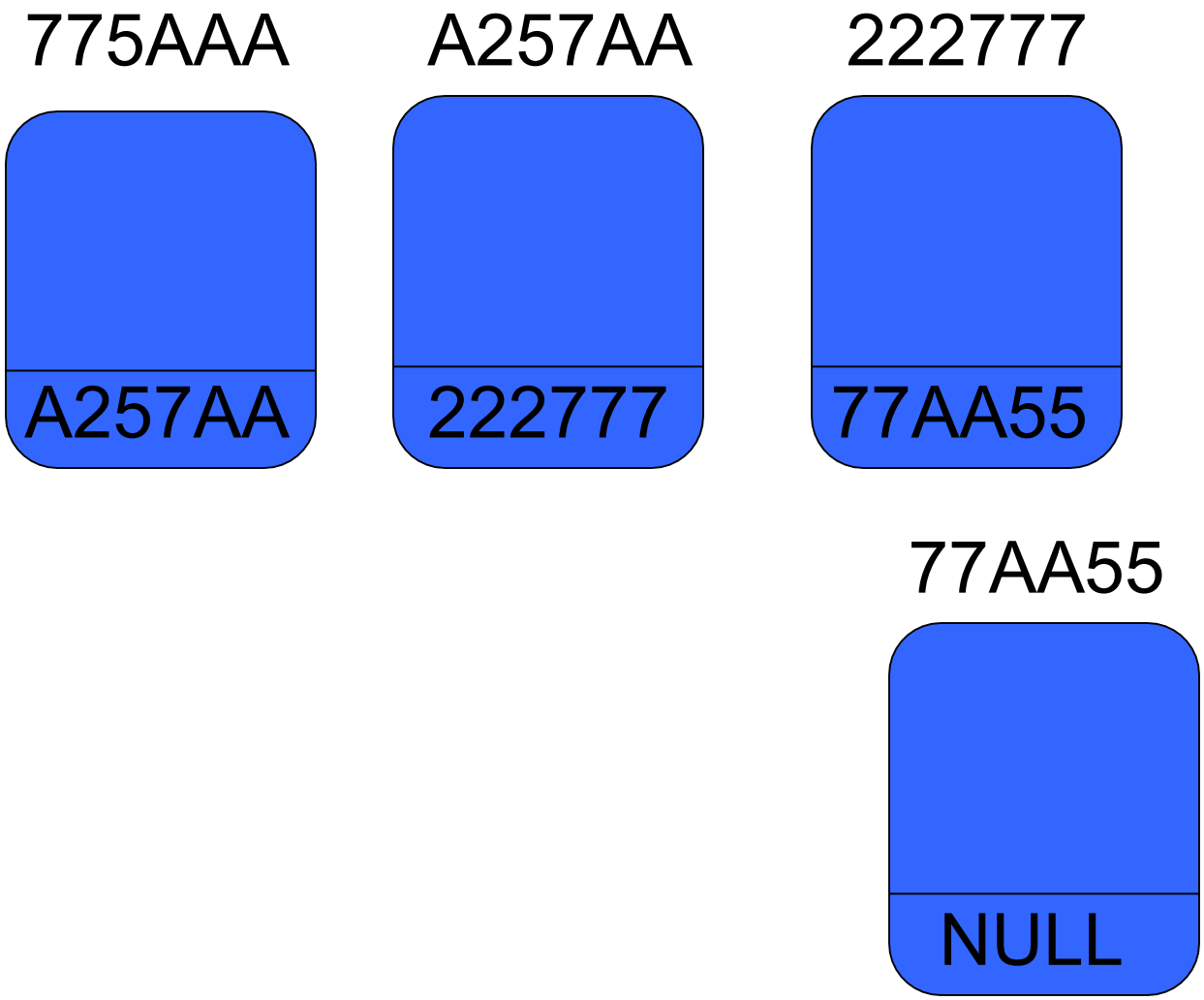


Link structure

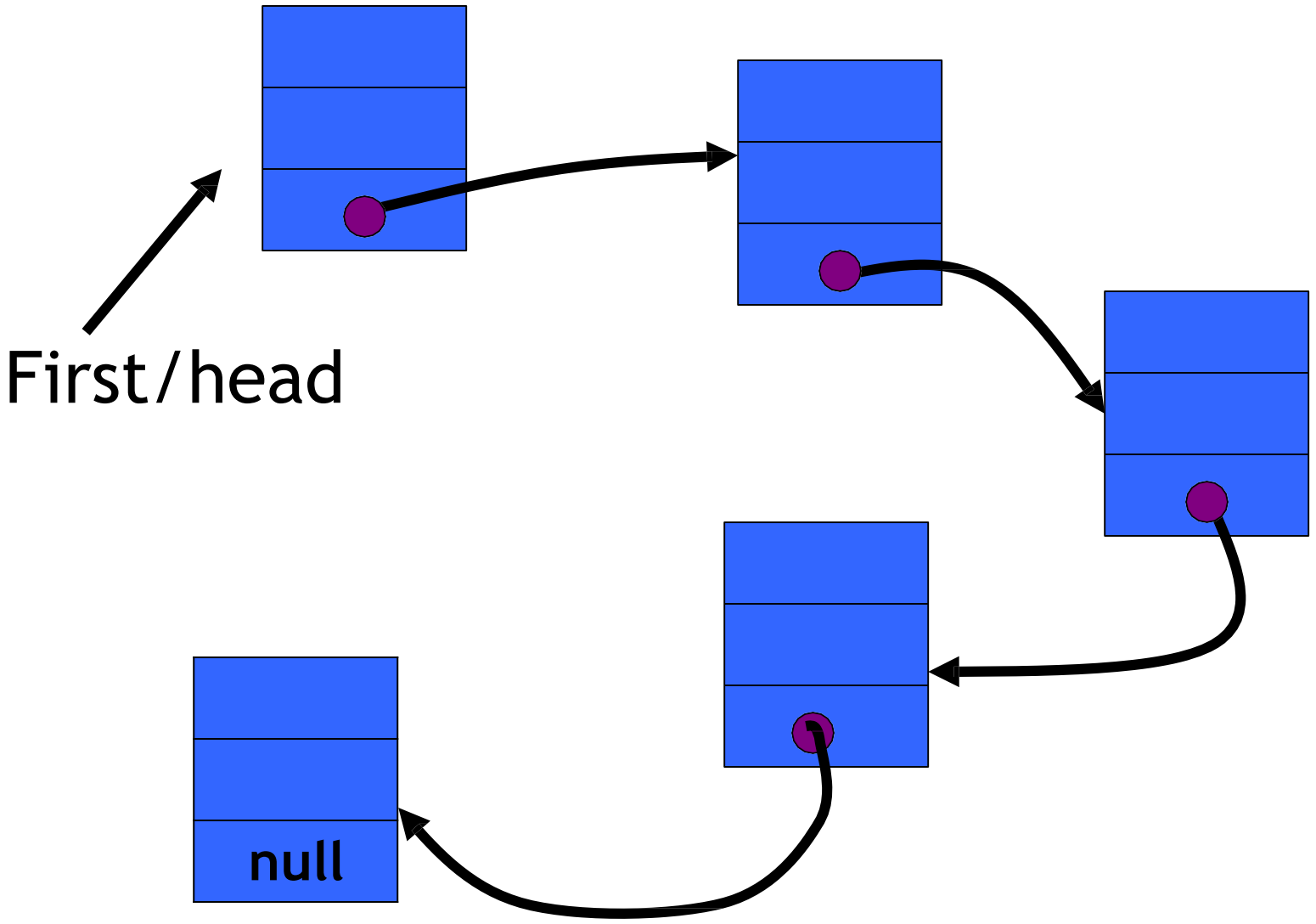


Linked List keeps a pointer of first Link!!!

First/head =775AAA



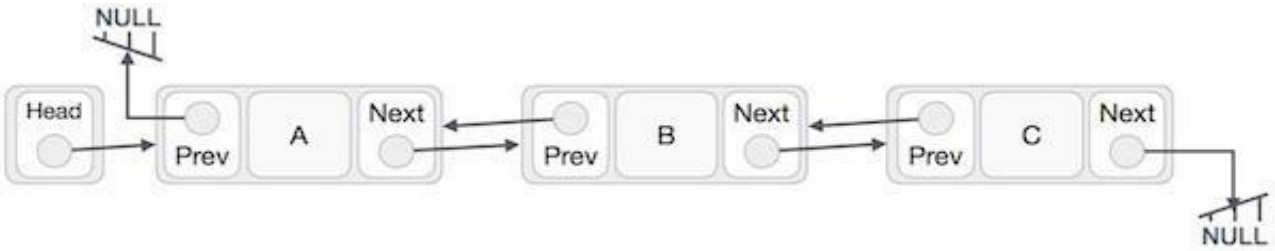
Linked List



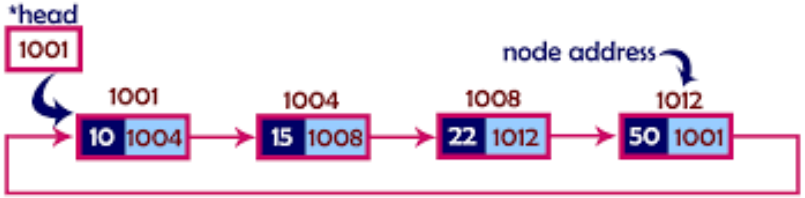
Types of Linked List

•**Simple Linked List** − Item navigation is forward only.

•**Doubly Linked List** − Items can be navigated forward and backward.



•**Circular Linked List** − Last item contains link of the first element as next and the first element has a link to the last element as previous.



Linked List Operations

- Create a node and linked list
- Traversal
- Search node
- Insert node

  at beginning, at end, after/before a node
- Delete node

  at beginning, at end, after/before a node
- Sort

How to create nodes

A node is a structure data type, can be created in two methods, statically and dynamically.

- Static method
  - use array of structures
  - declared as globally outside functions
  - declared locally within a function

- Dynamic method (mostly used for linked list)
  - use stdlib function malloc(size) get memory space

```
struct node *np = (struct node*) malloc(sizeof(struct node));
```

melloc() Function

- The function malloc() is used to allocate a certain amount of memory during the execution of a program.
- The malloc() function will request a block of memory from the heap.
- If the request is granted, the operating system will reserve the requested amount of memory.

Syntax:

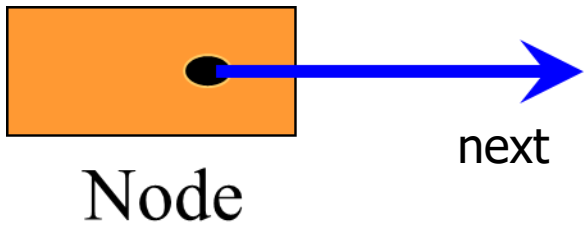
void *malloc(size_t size)

size − This is the size of the memory block in bytes.

List node declaration

- Each node contains a data element and a pointer to the next node in the list.

```
struct Node {
int data;
struct Node *next;
};
```



1) The structure contains data and a pointer

2) Data contains the data

3)next is the pointer to store the address of next node in the list.

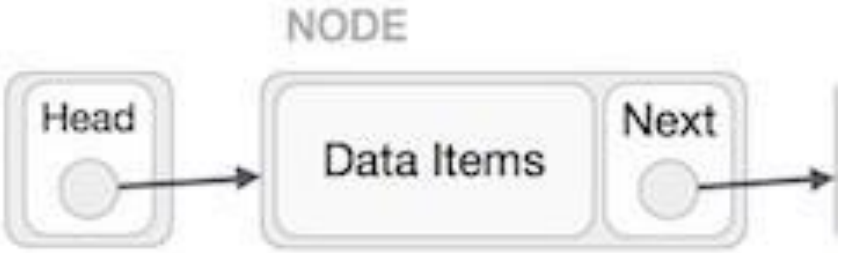
Self –referencing pointer.
Here next is a part of a node and it will point to the next node

Why we need pointers in Linked List?

- In case of linked list, data elements are allocated memory at runtime.
- Hence the memory location can be anywhere.
- Therefore to be able to access every node of the linked list, address of every node is stored in the previous node, hence forming a link between every node.
- In array, memory is allocated in contiguous manner.

Header Nodes

A header node is an extra node in the linked list that holds no data serves to satisfy the requirement that every node that contains an item have a previous node in the list.



Implementing Linked lists

```
struct node
{
  int data;
  struct node *next;
};
```

- Each struct node has a data item and a pointer to another struct node.



Implementing Linked lists

```
/* Initialize nodes */
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));

/* Assign data values */
one->data = 1; //(one->data  can be written as (*one).data)
two->data = 2;
three->data=3;

/* Connect nodes */
one->next = two;
two->next = three;
three->next = NULL;

/* Save address of first node in head */
head = one;
```



Implementing link list

if you wanted to put an element 4 between 1 and 2, the steps would be:

- Create a new struct node and allocate memory to it.
- Add its data value as 4
- Point its next pointer to the struct node containing 2 as data value
- Change next pointer of "1" to the node we just created.

Implementing Linked lists

- Add a variable to the end of the list, we can just continue advancing to the next pointer:

```
node_t * head = NULL;
head = malloc(sizeof(node_t));
head->val = 1;
head->next = malloc(sizeof(node_t));
head->next->val = 2;
head->next->next = NULL;
```

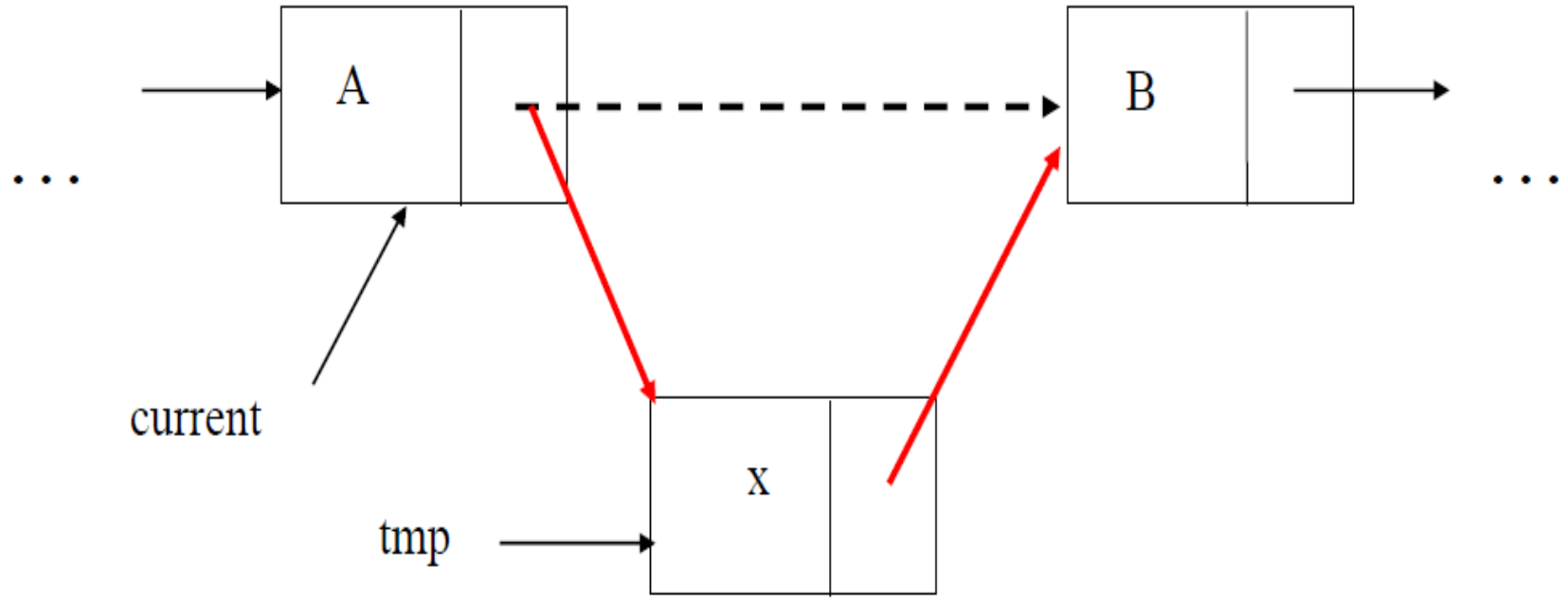


Inserting a node into a list

There are many ways you might want to insert a new node into a list:

- As the new first element
- As the new last element
- Before a given node
- After a given node
- Before a given value
- After a given value

Insertion

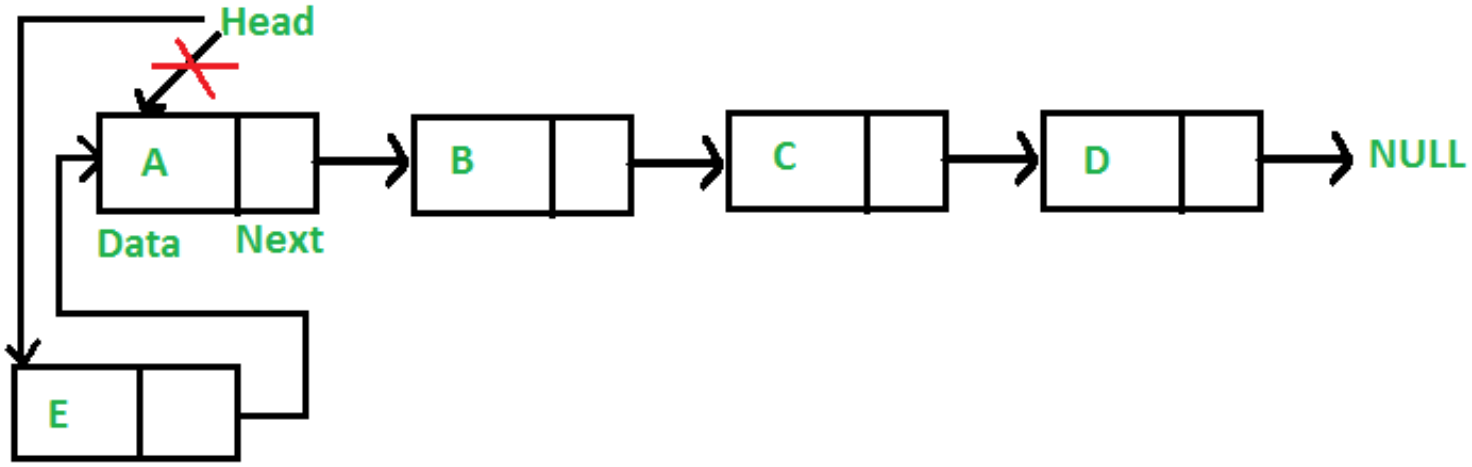



```
tmp = new Node;
tmp->data = x;
tmp->next = current->next;
current->next = tmp;
```

Inserting a node to the top of a list

4 steps process

- Create the new node, lets say E.
- Set the nodes data field (E→data = data).
- Set the next pointer of new node to head pointer (E→next = head).
- Set the head to point to the new node (head = E).



Inserting a node to the top of a list

```
/* Given a reference (pointer to pointer) to the head of a list
   and an int, inserts a new node on the front of the list. */

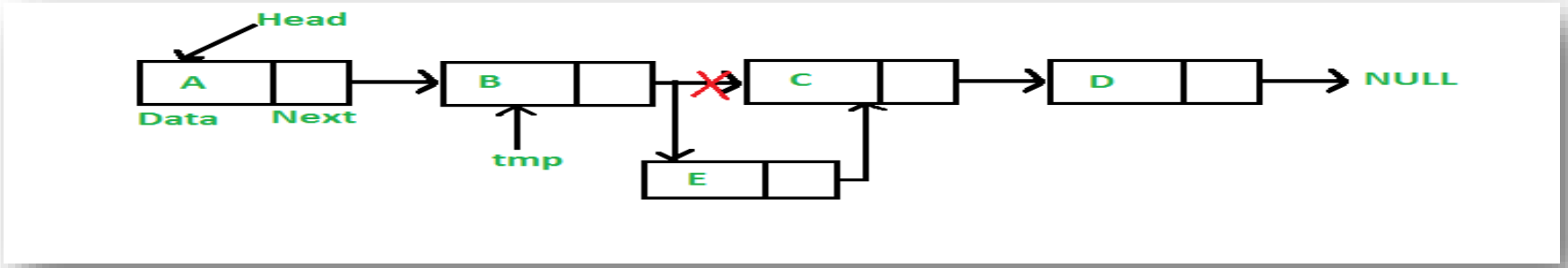

void insert(struct Node** head_ref, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));

    /* 2. put in the data  */
    new_node->data       = new_data;

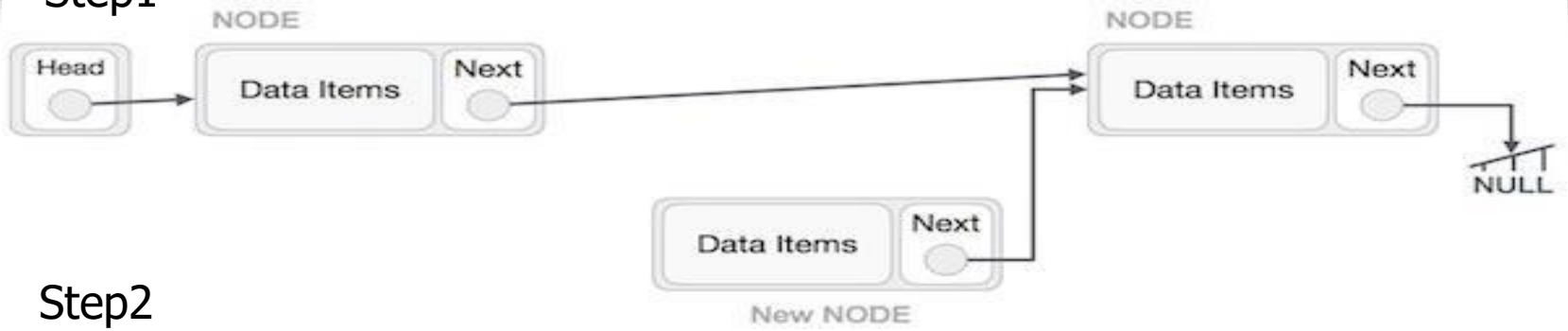
    /* 3. Make next of new node as
    head */
    new_node->next =(*head_ref);

    /* 4. move the head to point to the new node */
    (*head_ref)  = new_node;
}
```

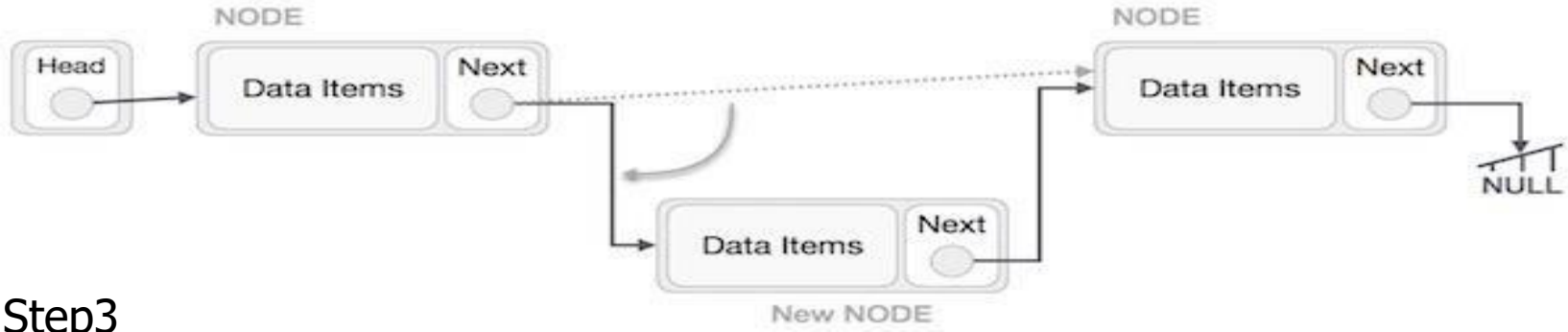
Add a node after a given node:



Step1



Step2



Step3



Add a node after a given node:

```
/* Given a node prev_node, insert a new node after the given
   prev_node */
void insertAfter(struct Node* prev_node, int new_data)
{
    /*1. check if the given prev_node is NULL */
    if (prev_node == NULL)
    {
      printf("the given previous node cannot be NULL");
      return;
    }

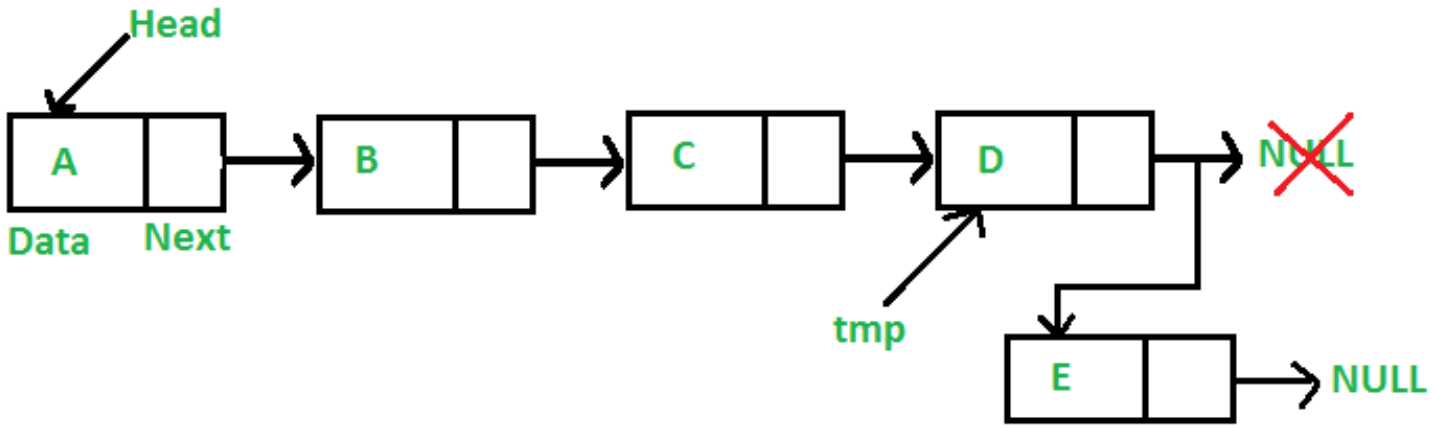
    /* 2. allocate new node */
    struct Node* new_node =(struct Node*) malloc(sizeof(struct Node));

    /* 3. put in the data */
    new_node->data = new_data;

    /* 4. Make next of new node as next of prev_node */
    new_node->next = prev_node->next;

    /* 5. move the next of prev_node as new_node */
    prev_node->next = new_node;
}
```

Add a node at the end



Traverse the list till end and then change the next of last node to new node.

Add a node at the end

```
/* Given a reference (pointer to pointer) to the head
   of a list and an int, appends a new node at the end */
void append(struct Node** head_ref, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*)
malloc(sizeof(struct Node));



    struct Node *last = *head_ref; /* used in step 5*/

    /* 2. put in the data */
    new_node->data = new_data;



    /* 3. This new node is going to be the last node, so
make next
          of it as NULL*/
    new_node->next = NULL;
```

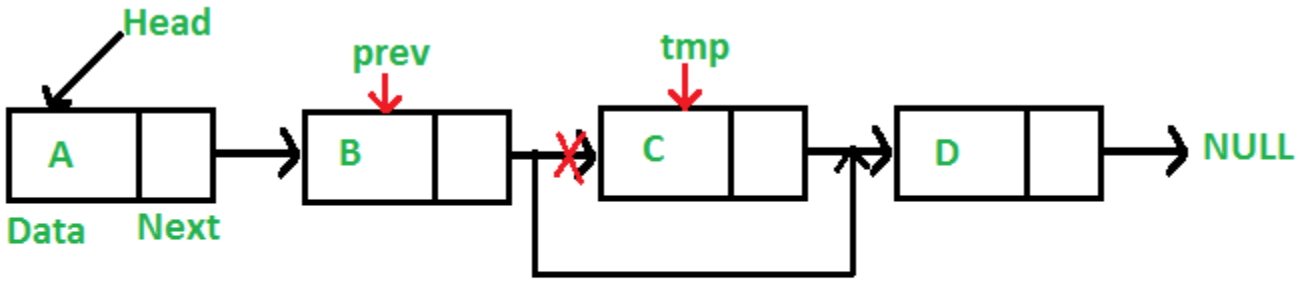
```
/* 4. If the Linked List is empty, then
make the new node as head */
    if (*head_ref == NULL)
    {
    *head_ref = new_node;
    return;
    }
/* 5. Else traverse till the last node */
while (last->next != NULL)
     last = last->next;


/* 6. Change the next of last node */
last->next = new_node;
return;     }
```

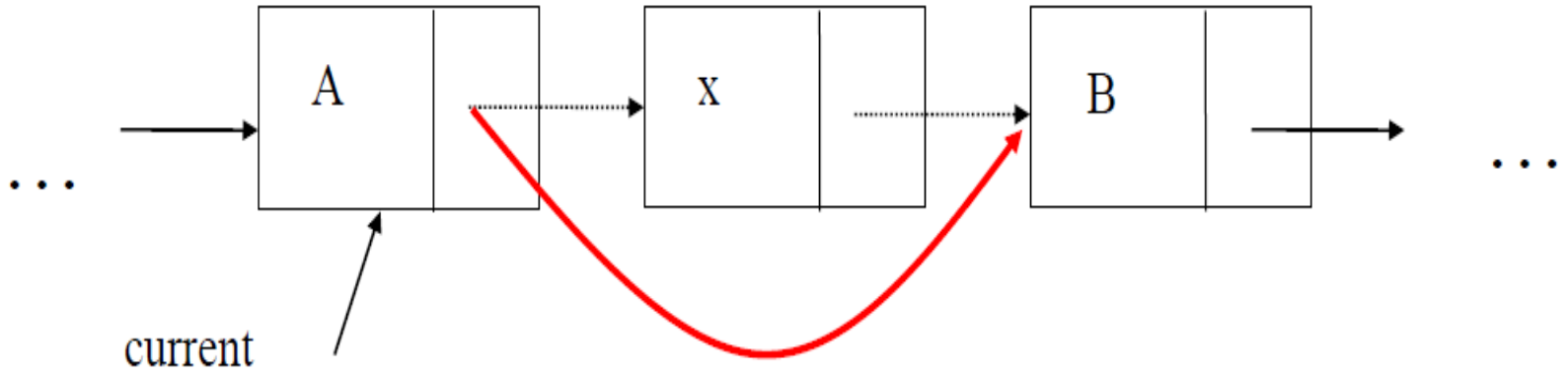
Deleting a node

To delete a node from linked list, we need to do following steps.

1) Find previous node of the node to be deleted.
2) Changed next of previous node.
3) Free memory for the node to be deleted.

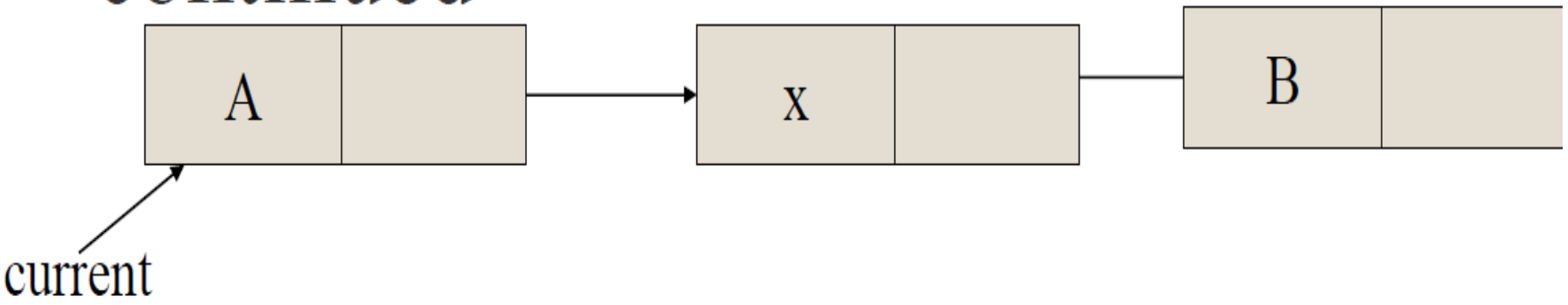


Deletion from a linked list



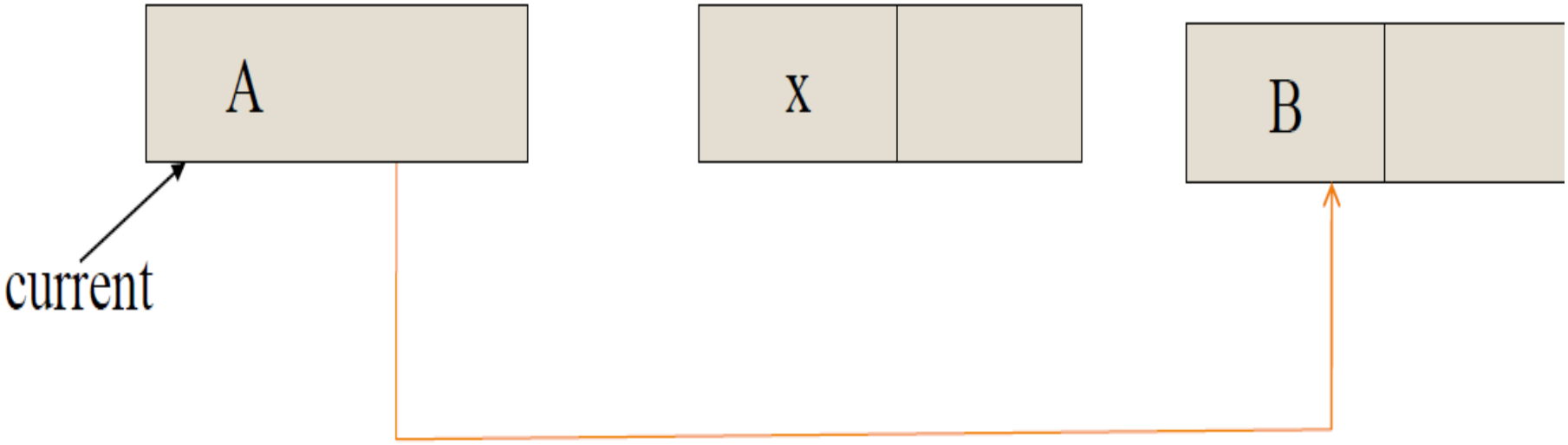

```
current->next = current->next->next;
```

Deletion from a linked list …… continued



To remove element x from a linked list; we set current to be the node prior to x and then have current's next reference by pass x.

current.next=current.next.next



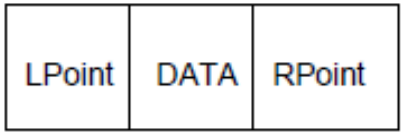
After Deletion

The list A,X,B now appear as A,B

Applications of Linked Lists

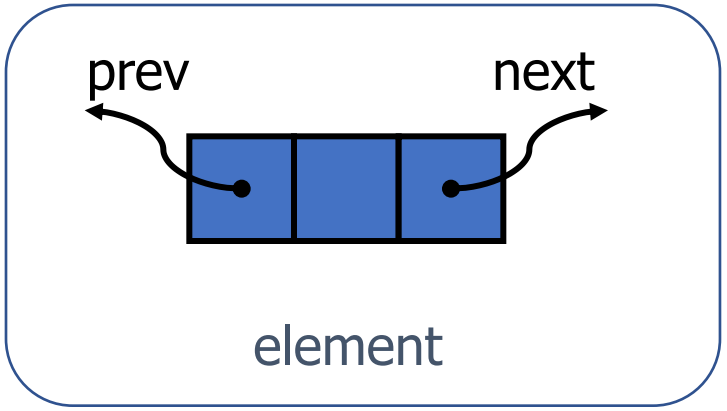
- Linked lists are used to implement stacks, queues, graphs, etc.
- Linked lists let you insert elements at the beginning and end of the list.
- In Linked Lists we don't need to know the size in advance.

Doubly Linked List (DLL)



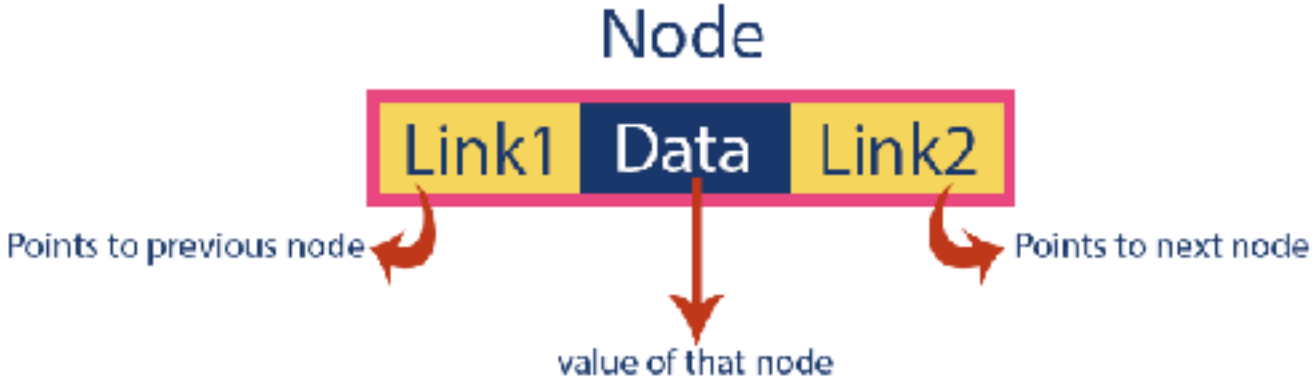
A typical doubly linked list node

- A doubly linked list is one in which all nodes are linked together by multiple links
- It is a way of going both directions in a linked list, forward and reverse.
- Every nodes in the doubly linked list has three fields:
  1. LeftPointer (prev)
  2. RightPointer (next)
  3. DATA.



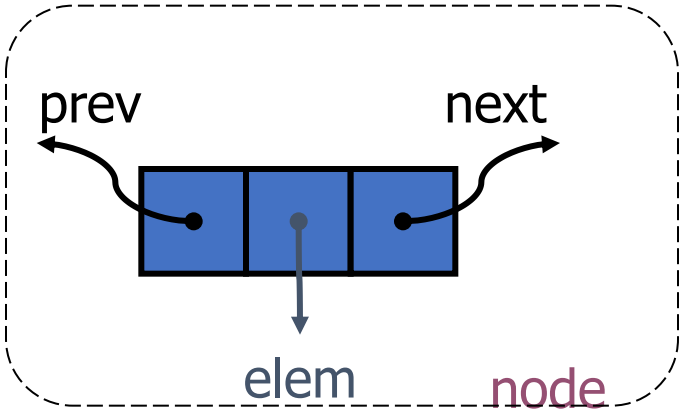
WHY DOUBLY LINKED LIST

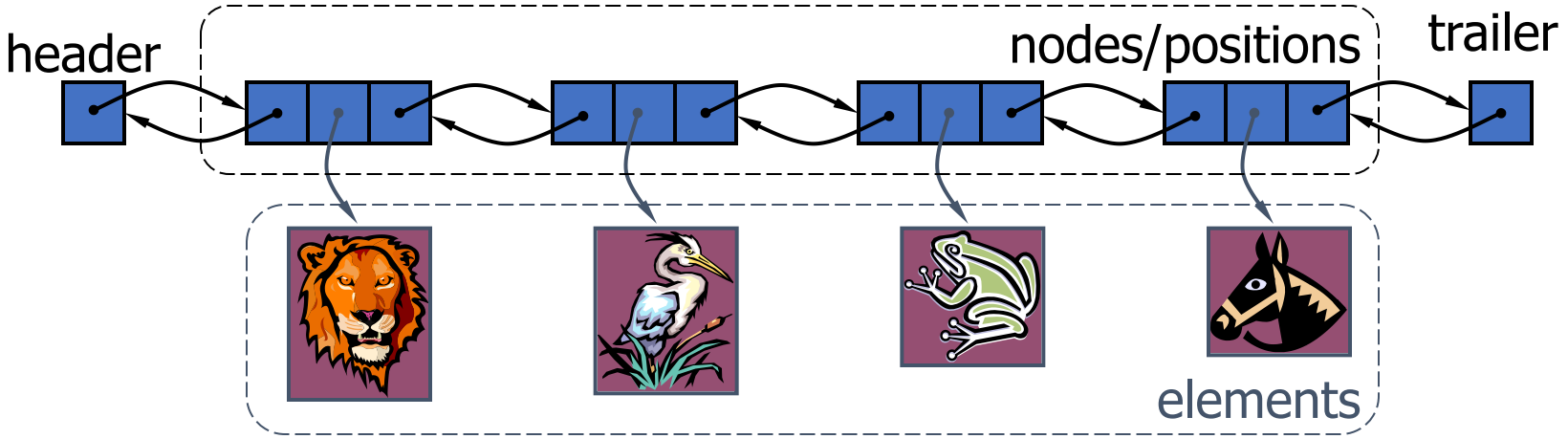
- If we have a problem in which moving in either direction is often necessary, then it is useful to have doubly linked lists.
- Many applications require a quick access to the predecessor node of some node in list.
- Each node now has two link data members,
  - One linking in the forward direction
  - One in the backward direction



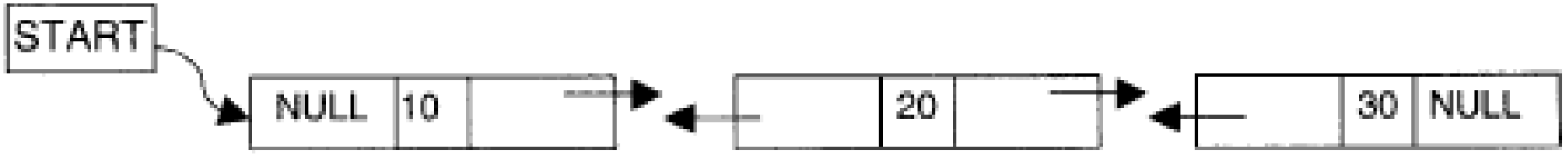
Doubly Linked List (DLL)

- A doubly linked list provides a natural implementation of the List ADT
- Nodes implement Position and store:
  - **element**
  - **link to the previous node**
  - **link to the next node**
- Special trailer and header nodes





Doubly Linked List (DLL)

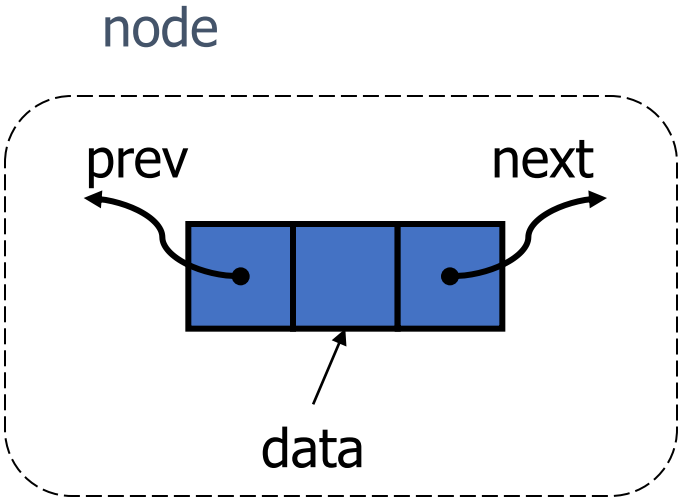


Doubly Linked List



Memory Representation of Doubly Linked List

Implementation




```
typedef struct node {
Int data;
struct node* next;
struct node* prev;
} node;
```

Advantages over Singly-linked Lists

- A DLL can be traversed in both forward and backward direction.
- Quick update operations:

  such as: insertions, deletions at *both* ends (head and tail), and also at the middle of the list.
- The delete operation in DLL is more efficient if pointer to the node to be deleted is given.

In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using previous pointer..

Disadvantages over singly linked list

- Every node of DLL Require extra space for an previous pointer. It is possible to implement DLL with single pointer though.
- All operations require an extra pointer previous to be maintained.

For example: in insertion, we need to modify previous pointers together with next pointers.

For example in following functions for insertions at different positions, we need 1 or 2 extra steps to set previous pointer.

Sentinel Nodes

- To simplify programming, two special nodes have been added at both ends of the doubly-linked list.
- Head and tail are dummy nodes, also called sentinels, do not store any data elements.
- Head: header sentinel has a ***null-prev*** reference (link).
- Tail: trailer sentinel has a ***null-next*** reference (link).

What we see from a Doubly-linked List?

A doubly-linked list object would need to store the following:

1. Reference to sentinel head-node;
2. Reference to sentinel tail-node; and
3. Size-counter that keeps track of the number of nodes in the list (excluding the two sentinels).

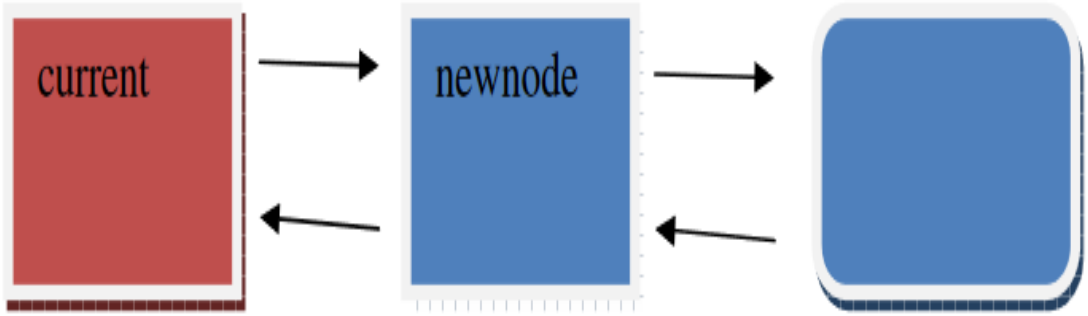
Basic Operations on a Doubly-Linked List

1. Add a node.
2. Delete a node.
3. Search for a node.
4. Traverse (walk) the list. Useful for counting operations or aggregate operations.

- Note: the operations on a doubly-linked list are exactly the same that are required for a singly-linked list.

Inserting to a Doubly Linked Lists

Suppose a new node, new node needs to be inserted after the node current.



Adding Nodes to a Doubly-Linked List

Adding a Node

There are four steps to add a node to a doubly-linked list:

- Allocate memory for the new node.
- Determine the insertion point to be after (pCur).
- Point the new node to its successor and predecessor.
- Point the predecessor and successor to the new node.

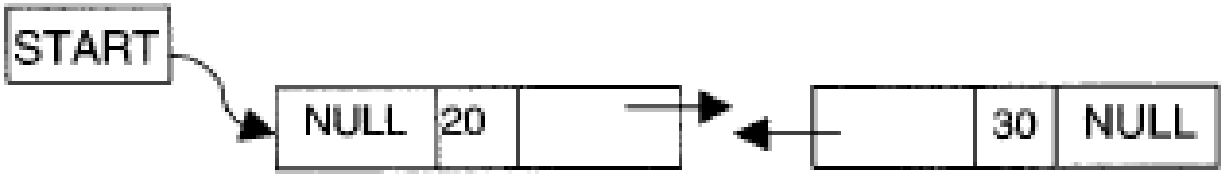
Current node pointer (pCur) can be in one of two states:

- it can contain the address of a node (i.e. you are adding somewhere after the first node – in the middle or at the end)
- it can be NULL (i.e. you are adding either to an empty list or at the beginning of the list)

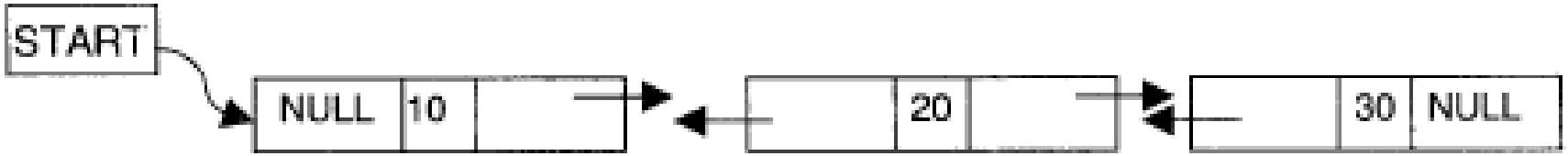
INSERTION



Add(20)

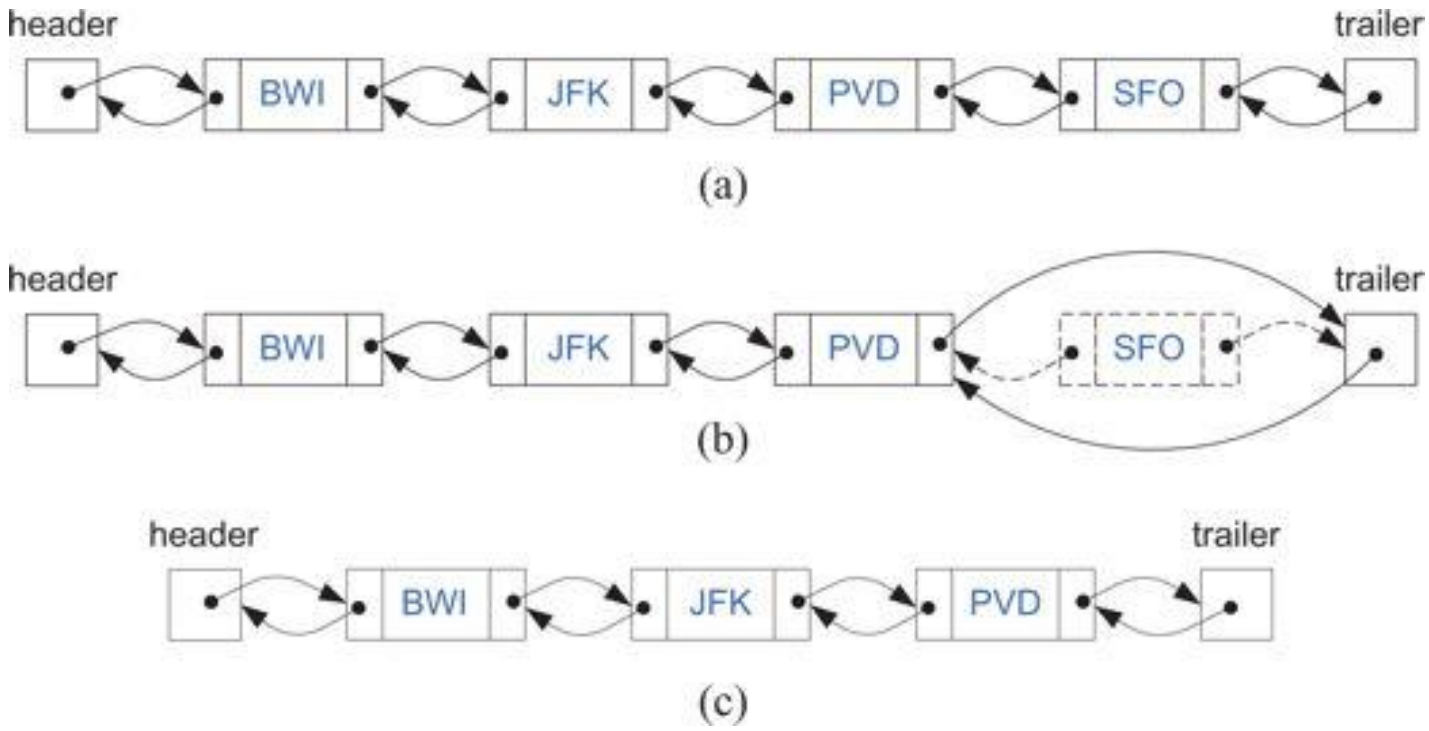


Insert (30) at the end



Insert (10) at the beginning

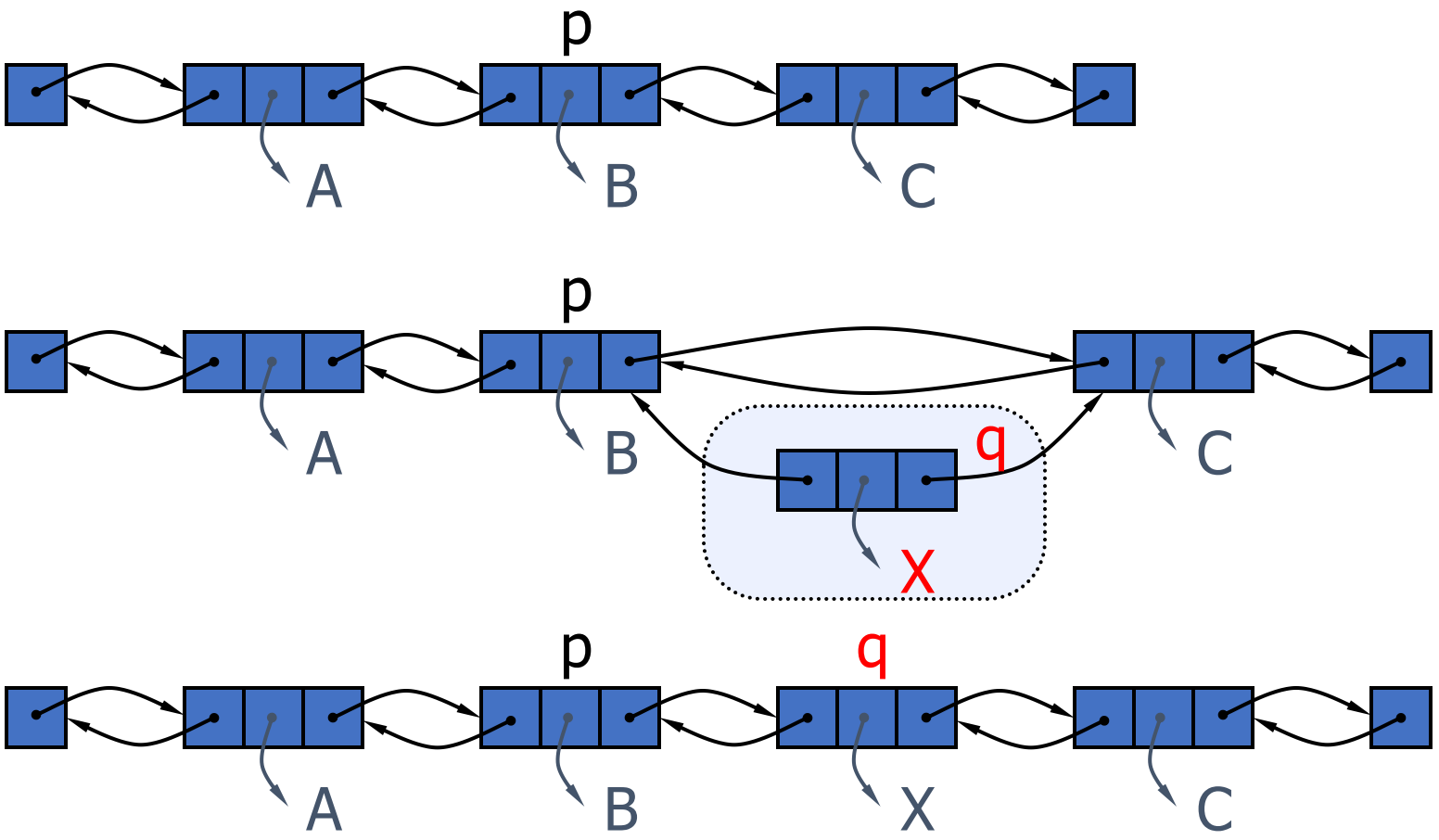
Removal from a Doubly-Linked List



Notice that before removal, we must check for empty list. If not, we will remove the last node in the list, as shown in Figure above.

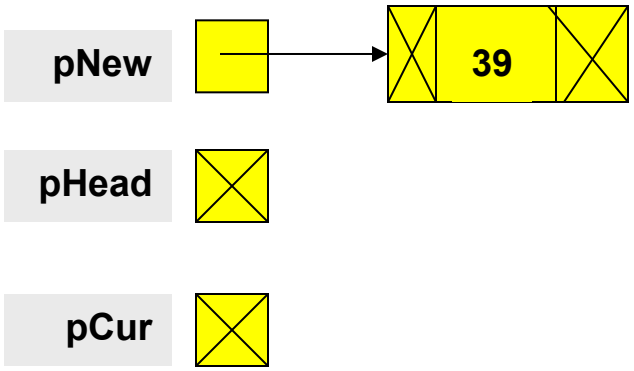
Insertion

- We visualize operation AddAfter(p, X), which returns position q



Adding Nodes to an Empty Doubly-Linked List

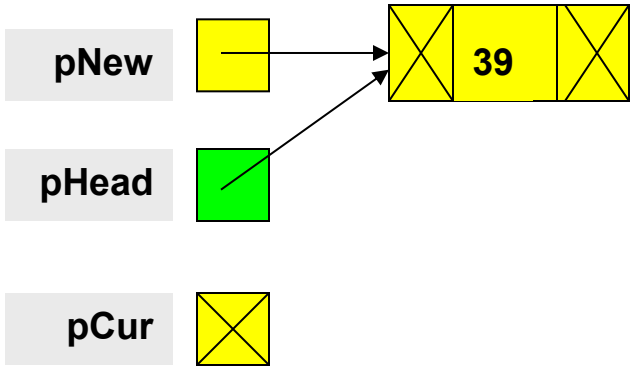
Initial:



Code:

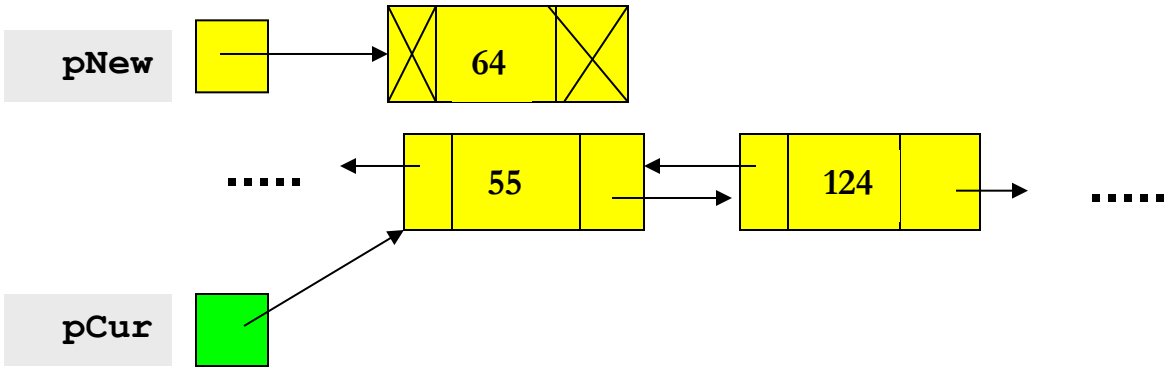
```
pNew = (struct node *)    /*create node*/
malloc(sizeof(struct dllnode));
New -> data = 39;
pNew -> right = pHead;
pNew -> left = pHead;
pHead = pNew;
```

After:

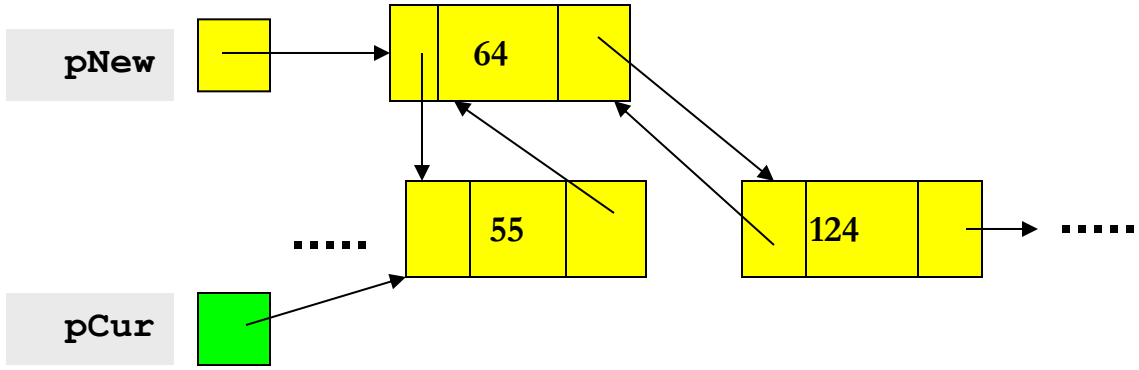


Adding a Node to the Middle of a Doubly-Linked List

Before:



After:



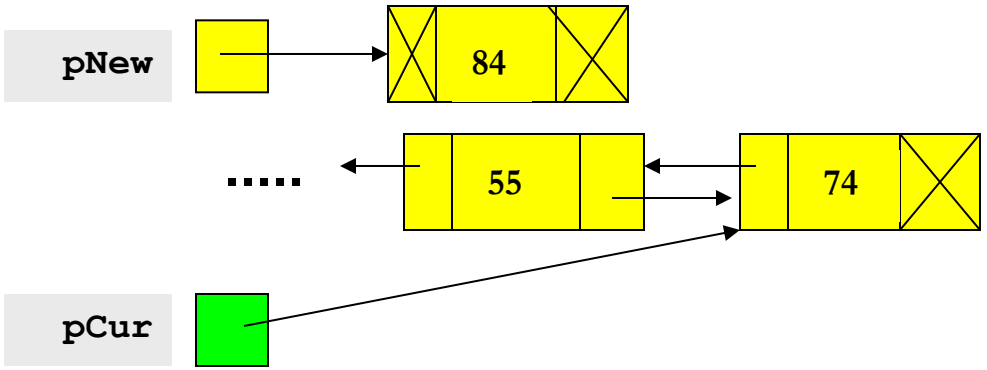
Adding a Node to the Middle of a Doubly-Linked List

The code

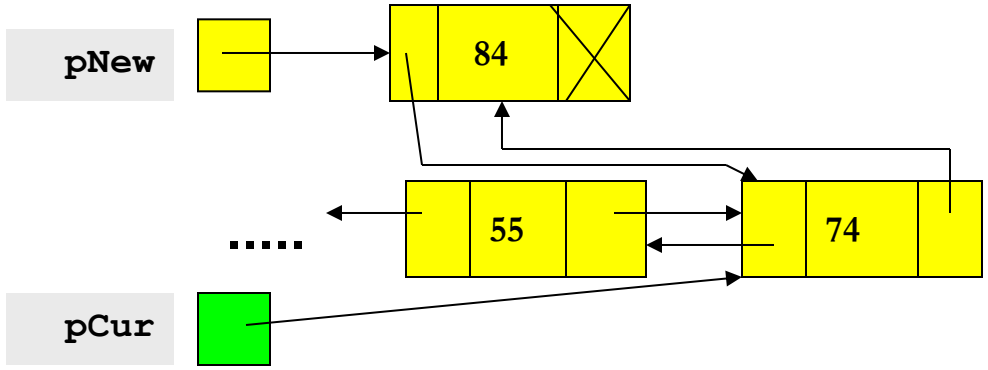
```
pNew = (struct node *) malloc(sizeof(struct dllnode));
pNew -> data = 64;
pNew -> left = pCur;
pNew -> right = pCur -> right;
pCur -> right -> left = pNew;
pCur -> right = pNew;
```

Adding a Node to the End of a Doubly-Linked List

Before:



After:



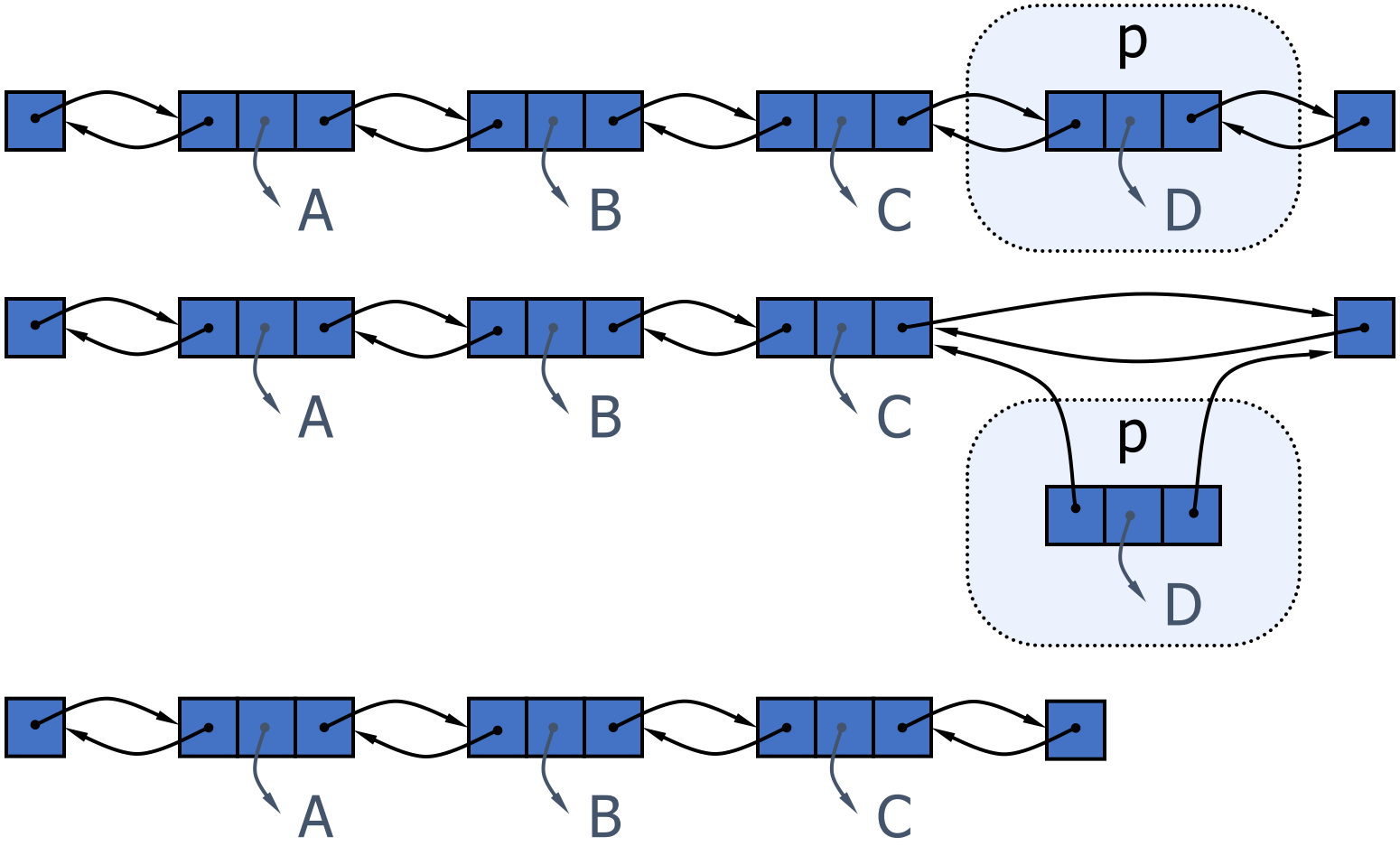
Adding a Node to the end of a Doubly-Linked List

The Code

```
pNew = (struct node *) malloc(sizeof(struct dllnode));
pNew -> data = 84;
pNew -> left = pCur;
pNew -> right = pCur -> right;
pCur -> right  = pNew;
```

Deletion

- We visualize remove(p), where p = last()

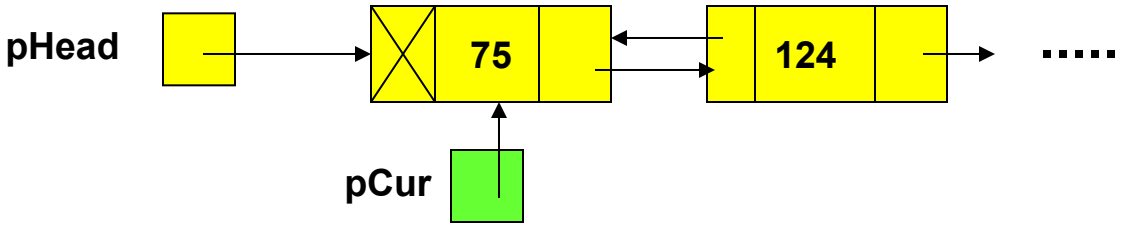


Deleting a Node from a Doubly-Linked List

- Deleting a node requires that we logically remove the node from the list by changing various links and then physically deleting the node from the list (i.e., return it to the heap).
- Any node in the list can be deleted. Note that if the only node in the list is to be deleted, an empty list will result. In this case the head pointer will be set to NULL.
- To logically delete a node:
  - First locate the node itself (pCur).
  - Change the predecessor's and succesor's link fields to point each other (see example).
  - Recycle the node using the free() function.

Deleting the First Node from a Doubly-Linked List

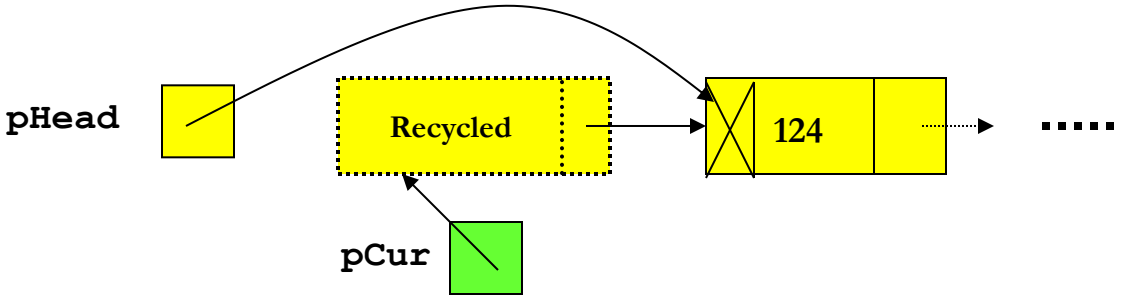
Before:



Code:

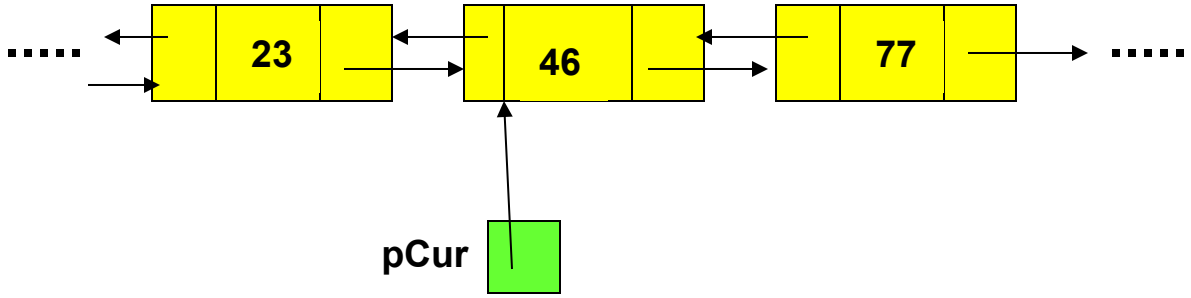
```
pHead = pCur -> right;
pCur ->right -> left = NULL;
free(pCur);
```

After:

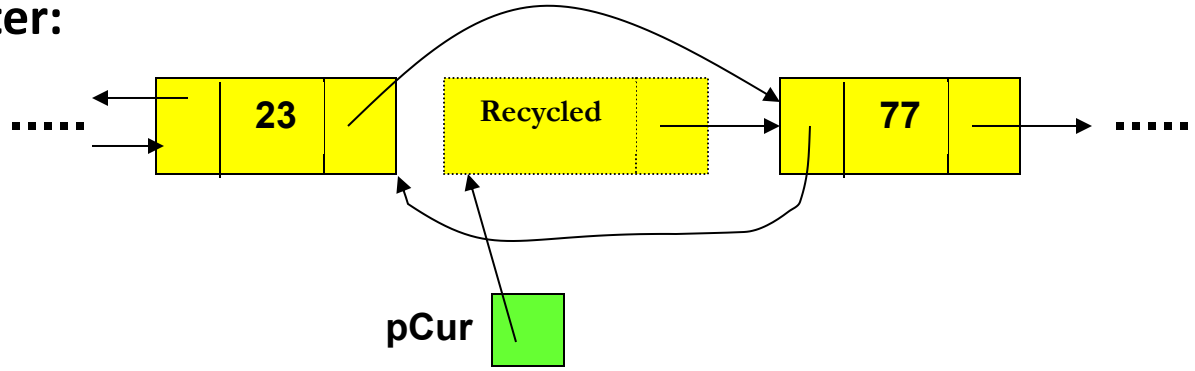


Deleting a Node from a Linked List – General Case

Before:



After:



Deleting a Node From a Doubly-Linked List

The Code

```
//delete a node from a linked list
if (pCur -> left == NULL){
        //deletion is on the first node of the list
     pHead = pCur -> right;
     pCur -> right -> left = NULL;
{
else {
        //deleting a node other than the first node of the list
       pCur -> left -> right = pCur -> right;
    pCur -> right -> left = pCur -> left;
}
free(pCur).
```

Searching a Doubly-Linked List

- Notice that both the insert and delete operations on a linked list must search the list for either the proper insertion point or to locate the node corresponding to the logical data value that is to be deleted.

```
//search the nodes in a linked list
pCur = pHead;
//search until the target value is found or the end of the list is reached
while (pCur != NULL && pCur -> data != target) {
     pCur = pCur -> right;
}
//determine if the target is found or ran off the end of the list
if (pCur != NULL)
  found = 1;
else
  found = 0;
```

END